off the end, by the use of a control flow statement such as `break` or `return`, or by throwing an exception. The wait functions of `std::condition_variable` require an instance of `std::unique_lock<std::mutex>`, and all instantiations of `std::unique_lock` are suitable for use with the `Lockable` parameter for the `std::condition_variable_any` wait functions.

If the supplied `Mutex` type meets the `Lockable` requirements, then `std::unique_lock<Mutex>` also meets the `Lockable` requirements. If, in addition, the supplied `Mutex` type meets the `TimedLockable` requirements, then `std::unique_lock<Mutex>` also meets the `TimedLockable` requirements.

Instances of `std::unique_lock` are `MoveConstructible` and `MoveAssignable` but not `CopyConstructible` or `CopyAssignable`.

Class definition

```
template <class Mutex>
class unique_lock
{
public:
    typedef Mutex mutex_type;

    unique_lock() noexcept;
    explicit unique_lock(mutex_type& m);
    unique_lock(mutex_type& m, adopt_lock_t);
    unique_lock(mutex_type& m, defer_lock_t) noexcept;
    unique_lock(mutex_type& m, try_to_lock_t);
    template<typename Clock,typename Duration>
    unique_lock(
        mutex_type& m,
        std::chrono::time_point<Clock,Duration> const&
absolute_time);

    template<typename Rep,typename Period>
    unique_lock(
        mutex_type& m,
        std::chrono::duration<Rep,Period> const& relative_time);

    ~unique_lock();
```